

# Software Architecture – Assignment 6

## Activity and BPMN Diagrams with Requirements Traceability

### Group Members:

Kashkenov Madiyar, Toktarov Nurislam

## 1. Introduction and Case Summary

### Selected System

The selected system is a **Geolocation-Based Service Worker Marketplace Platform**, consistently continued from Assignments 1–5.

### System Purpose and Main Users

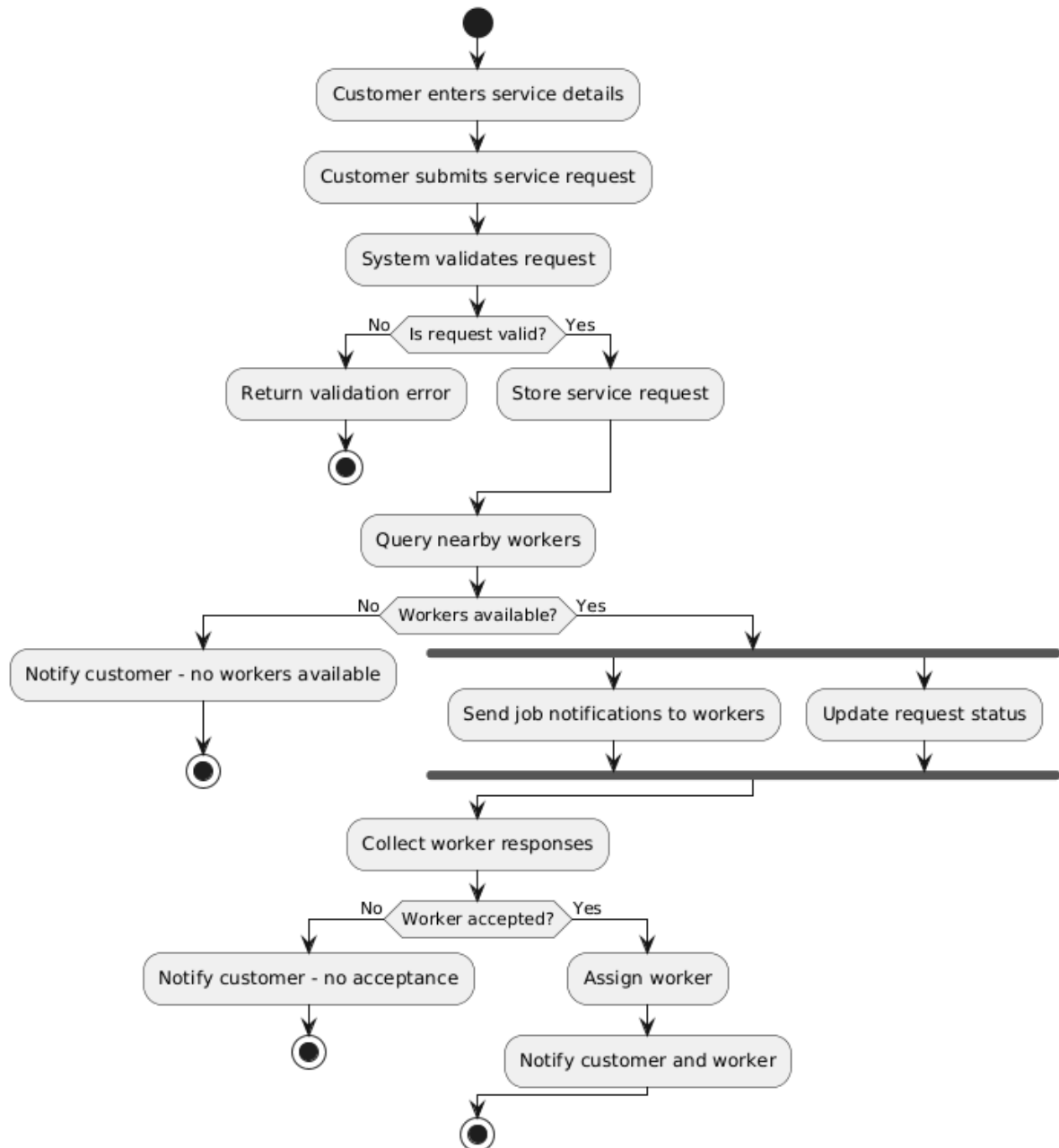
The platform connects customers who require immediate or scheduled services (e.g., plumbing, electrical work, cleaning) with nearby service workers using real-time geolocation. Customers can create service requests, track job progress, communicate with workers, and complete secure payments. Service workers manage profiles, receive job notifications, perform services, and track earnings. Administrators oversee verification, disputes, analytics, and compliance.

The system is designed as a **cloud-native microservices architecture**, optimized for **performance, scalability, security, and reliability**, as defined in Assignments 1 and 2 and justified through ADRs in Assignment 4.

## 2. Activity Diagrams

### Activity Diagram 1: Service Request Creation and Worker Matching

**Activity Diagram 1: Service Request Creation and Worker Matching**



#### Purpose of the Process

This activity diagram models how a customer creates a service request and how the system identifies and notifies nearby available service workers.

## Functional Scope

This process implements:

- **FR-1:** Customer can create a service request
- **FR-2:** System finds nearby available service workers
- **FR-3:** Workers receive real-time job notifications

## Step-by-Step Flow

1. Customer enters service details and submits a request.
2. System validates request data.
3. **Decision:** Is request data valid?
  - If no → error is returned to the customer.
4. System stores the service request.
5. System queries nearby workers using geolocation.
6. **Decision:** Are workers available?
  - If no → customer is notified about unavailability.
7. **Fork:**
  - Notify available workers (asynchronous)
  - Update request status
8. **Join:** System waits for worker responses.

## Parallel and Alternative Paths

- Notifications and request status updates occur in parallel.
- Alternative paths handle invalid input and lack of available workers.

## Non-Functional Requirements Traceability

- **Performance:** Real-time matching and notifications (<2s)
- **Scalability:** Parallel notifications support high concurrency
- **Reliability:** Request persistence before notifications

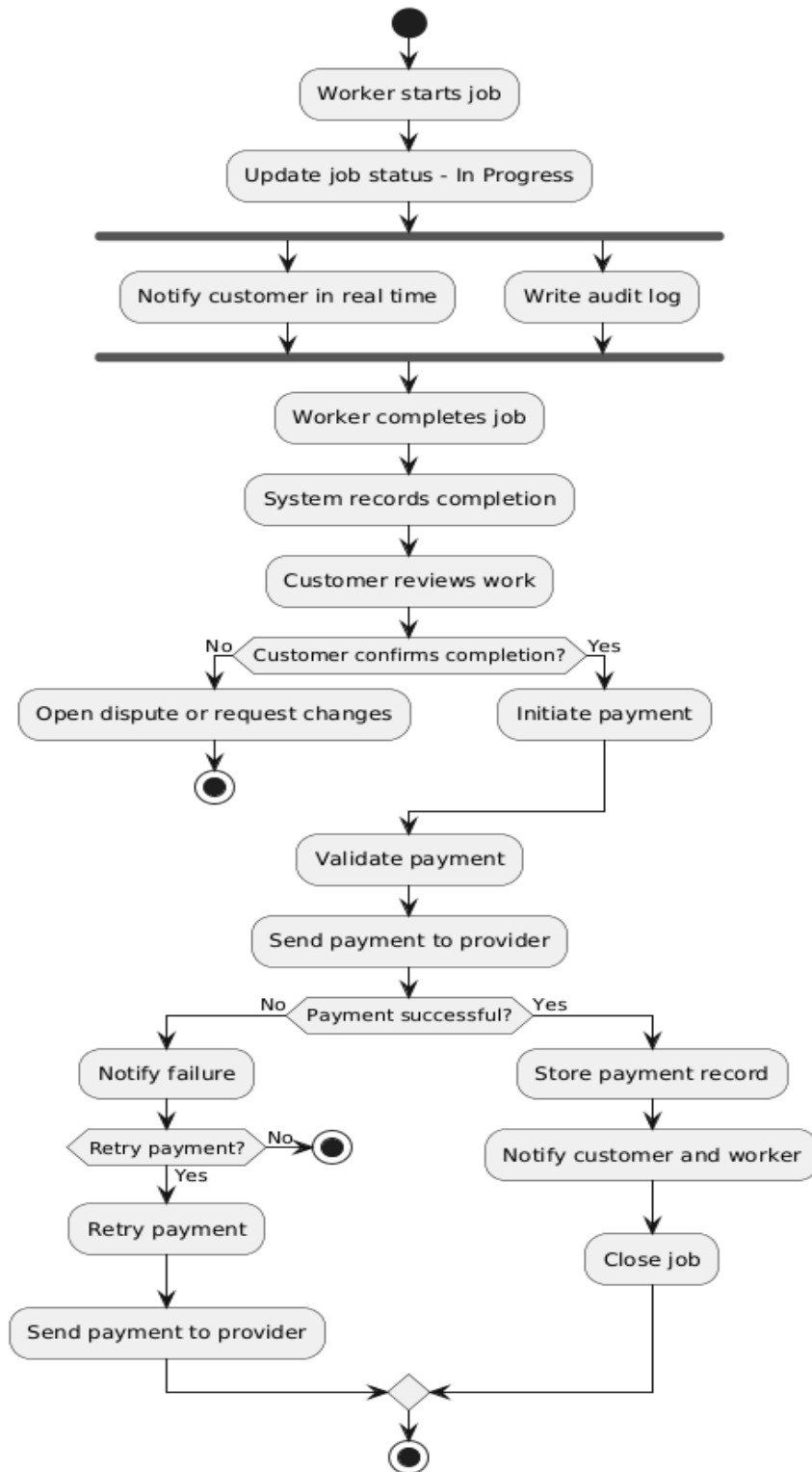
(From Assignment 2)

## Why This Level of Detail

The process includes validation, branching, and concurrency, reflecting real system behavior rather than a simple linear flow.

## Activity Diagram 2: Job Execution and Secure Payment Completion

Activity Diagram 2: Job Execution and Secure Payment



## Purpose of the Process

This diagram models job execution, completion confirmation, and payment processing.

## Functional Scope

This process implements:

- **FR-4:** Worker updates job status
- **FR-5:** Customer confirms job completion
- **FR-6:** Secure payment processing

## Step-by-Step Flow

1. Worker starts job and updates status.
2. System records job progress.
3. Worker marks job as completed.
4. Customer reviews and confirms completion.
5. **Decision:** Is job accepted by customer?
  - If no → dispute process initiated.
6. System initiates payment.
7. **Decision:** Payment successful?
  - If no → retry or failure notification.
8. Payment is stored and confirmations are sent.

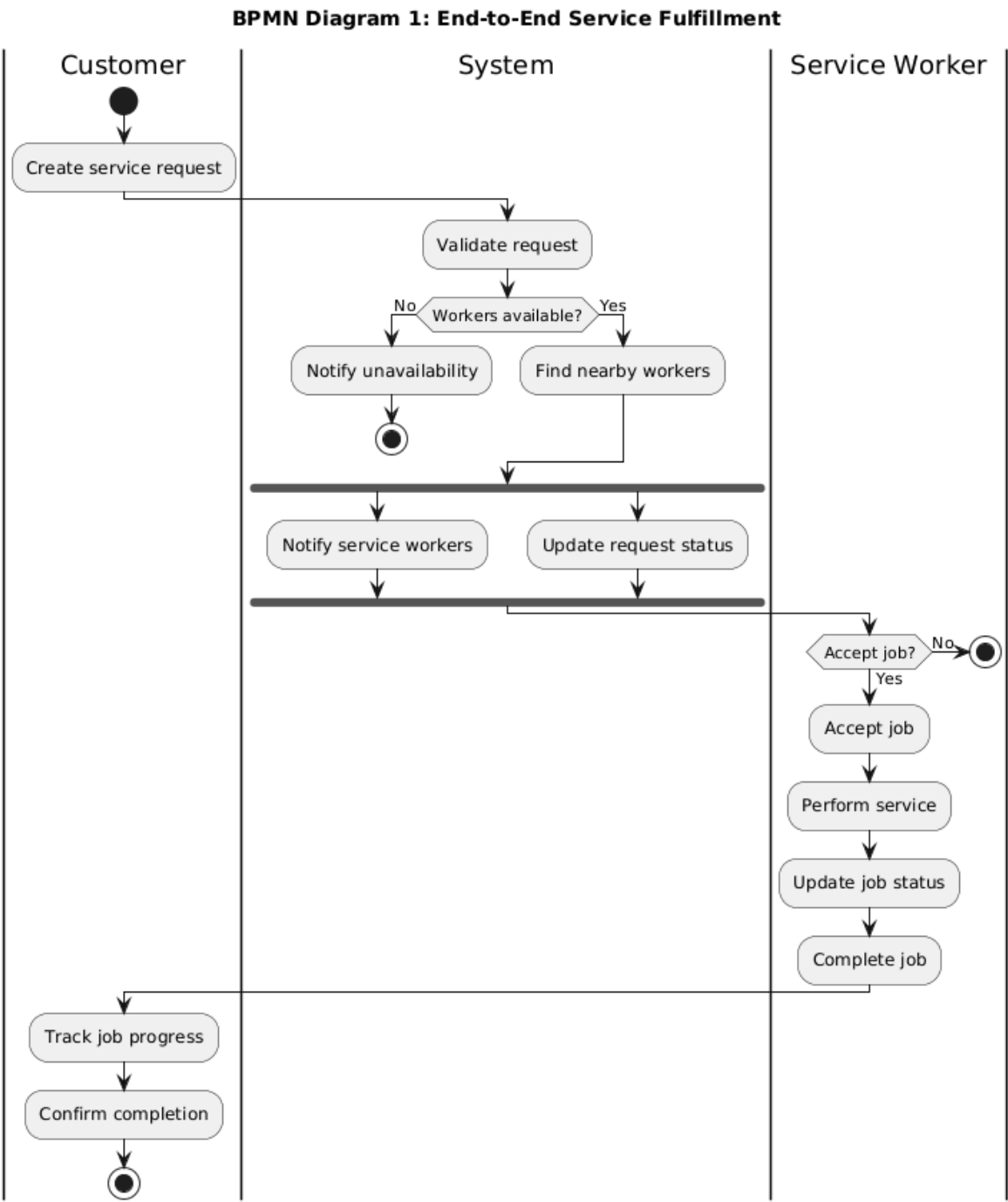
## Non-Functional Requirements Traceability

- **Security:** Payment handled through isolated Payment Service (PCI-DSS)
- **Reliability:** Payment only after job completion
- **Auditability:** All transitions are logged

## Why This Level of Detail

Quality attributes such as security and reliability directly influence control flow (approval, retries, confirmation).

### 3. BPMN Diagrams



### BPMN Diagram 1: End-to-End Service Fulfillment Process

#### Business Process

This BPMN diagram represents the full business workflow from service request to job completion.

## Participants (Pools / Lanes)

- Customer
- Service Marketplace System
- Service Worker

## Process Explanation

1. Customer submits service request.
2. System validates and publishes request.
3. **XOR Gateway:** Are workers available?
4. Worker accepts job.
5. Job is executed.
6. **Parallel Gateway:**
  - Worker updates status
  - Customer tracks progress
7. Job is completed and confirmed.
8. Process ends.

## Why BPMN (not Activity Diagram)

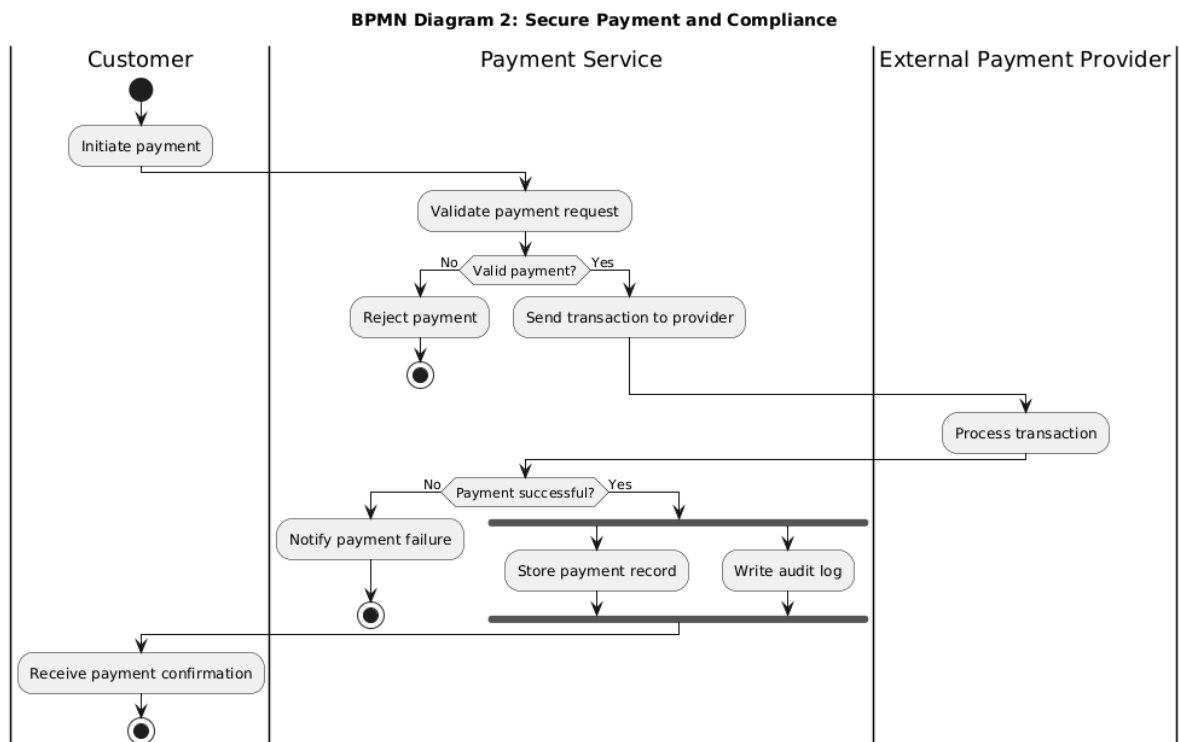
BPMN is used because:

- Multiple participants are involved
- Responsibilities must be clearly separated
- Business-level view is required

## Requirements Traceability

- **FR-1 to FR-5** (Assignment 1)
- **Availability & Performance NFRs** (Assignment 2)

# BPMN Diagram 2: Secure Payment and Compliance Process



## Business Process

This BPMN diagram models secure payment handling and compliance-driven steps.

## Participants

- Customer
- Payment Service
- External Payment Provider
- Administrator (audit role)

## Process Explanation

1. Customer initiates payment.
2. Payment Service validates transaction.
3. External provider processes payment.
4. **XOR Gateway:** Payment success?
5. **Parallel Gateway:**
  - Notify customer and worker
  - Store audit record
6. Administrator can review payment logs.
7. Process ends.



## Requirements Traceability

- **FR-6:** Secure payment processing
- **NFR-Security:** PCI-DSS, encryption, audit trails
- **NFR-Reliability:** No payment without job completion

## Why BPMN

BPMN clearly expresses:

- Separation of duties
- Compliance and audit responsibilities
- Cross-system interaction

## 4. Consistency and Traceability

All processes align with:

- **Functional requirements** (Assignment 1)
- **Quality attributes** (Assignment 2)
- **Microservices architecture** (Assignment 4)
- **Components and interactions** (Assignment 5)

Terminology, responsibilities, and constraints remain consistent across all assignments.

## 5. Assumptions and Open Questions

### Assumptions

- Users agree to location sharing
- External payment providers are reliable and compliant
- Redis is sufficient for real-time messaging

### Open Questions

- Should premium users receive faster matching?
- When should Redis Pub/Sub be replaced by Kafka?
- How will future regulations affect payment data retention?